Traffic Flow Dynamics in the Wolfram Language

From phantom jams on a ring road to a full-city multi-agent simulation of Aberdeen — a self-contained port of classical traffic-flow models.

Marco Thiel, April 2026 — github.com/mthiel74/TrafficJAms

Abstract

Traffic jams are the canonical everyday example of an emergent phenomenon: each driver follows simple local rules, and the system as a whole spontaneously produces stop-and-go waves, phantom jams, shockwaves, and city-scale gridlock. Since the 1950s these have been modelled at wildly different scales — as a hyperbolic PDE (Lighthill–Whitham–Richards), as individual vehicles following an ODE (Intelligent Driver Model, Bando OVM), as a stochastic cellular automaton (Nagel–Schreckenberg), as a queueing network at intersections, or as a convex optimisation problem on a directed graph (Wardrop equilibrium). This notebook walks through all of those in turn, each with a short derivation, a Wolfram-Language implementation, and output from a real simulation. The running case study is Aberdeen in north-east Scotland — a city whose peculiar geography (squeezed between the Rivers Don and Dee, the coast, and the hills, with two critical bridges) makes it an unusually nice traffic-engineering laboratory. The final section applies the full pipeline to the 11 925-node Aberdeen drivable road network extracted from OpenStreetMap, with 600 agents, UK-standard 90 s signal cycles, and roundabout gap acceptance.

Figure 1. 600 vehicles on the full Aberdeen road network (11,925 drivable nodes, 26,453 directed edges after strongly-connected-component restriction). Vehicles follow the Intelligent Driver Model with hierarchy-weighted noisy routing and staggered entry; 151 OSM-tagged signalised junctions and ~1000 roundabout-tagged edges enforce intersection control. Colours encode instantaneous speed (green = free flow, red = stopped).

1. Why model traffic?

Traffic engineers need quantitative tools to answer questions like: What happens to queue lengths on Union Street if we add a bus gate? If the Bridge of Don closes for maintenance, does Anderson Drive absorb the flow, or does half the city seize up? What is the shortest green phase for the busy intersection at Mounthooly that still keeps average delay under 30 s? None of these questions has a closed-form answer, but all of them are tractable in simulation once the correct mathematical model has been picked for the right spatial scale.

There is no single "right" model of traffic; there is a hierarchy of models, each trading detail for tractability. At the coarsest end we treat traffic as a compressible fluid; at the finest end we simulate every vehicle with its own ODE. In between sit stochastic cellular automata and queueing approximations. This notebook introduces each of these scales, shows the canonical model for it, and ends by stitching them all together on the real Aberdeen street map.

Aberdeen is a good running example for three reasons: it is small enough to be tractable (pop. ~200 000, compact historic centre), large enough to be interesting (international airport, oil-industry commuter flows, a peripheral ring road), and its geography creates natural bottlenecks — the two

main river bridges, the Haudagain roundabout, the Anderson Drive corridor — that any model will want to reproduce.

2. The macroscopic view: the LWR model

Lighthill and Whitham (1955), followed independently by Richards (1956), proposed that traffic on a long road obeys the same conservation law as any other compressible one-dimensional fluid: vehicles cannot be created or destroyed, so the density $\rho(x,\ t)$ and flow $Q(\rho)$ satisfy

$$\frac{\partial \rho}{\partial t} + \frac{\partial Q(\rho)}{\partial x} = 0$$

This is a first-order hyperbolic PDE. The missing ingredient is the *fundamental diagram* — an empirical relation between density and flow. The simplest, due to Greenshields (1935), is linear in the speed:

$$Q(\rho) = \rho\, v_{\max}\left(1 - \frac{\rho}{\rho_{\max}}\right)$$

so that flow is zero both at empty road (ρ=0) and at bumper-to-bumper jam (ρ=$\rho_{\max}$), and is maximised at the critical density $\rho_c = \rho_{\max}/2$. Plugging this into the conservation law gives a nonlinear scalar PDE that develops shocks from smooth initial data: the mathematical origin of a traffic jam.

We discretise it with the Godunov finite-volume scheme (the simplest upwind flux that respects the shock structure). The numerical flux between cells with left density ρ_L and right density ρ_R is

$$Q^{\,G}_{i+1/2} = \text{if } \rho_L \le \rho_R : \min(Q(\rho_L), Q(\rho_R),\ Q(\rho_c))$$

with the usual case analysis for ρL > ρR. A CFL condition (Δt ≤ Δx / $v_{\max}$) keeps the scheme stable. The Wolfram implementation is a dozen lines:

```
Greenshields[rho_, rhoMax_, vMax_] := rho vMax (1 - rho / rhoMax);
GodunovFlux[rhoL_, rhoR_, rhoMax_, vMax_] := Module[{rhoCrit = rhoMax / 2,
  qL = Greenshields[rhoL, rhoMax, vMax],
    qR = Greenshields[rhoR, rhoMax, vMax]},
 Which[
  rhoL ≤ rhoR,
   Which[rhoL ≥ rhoCrit, qL, rhoR ≤ rhoCrit, qR,
     True, Greenshields[rhoCrit, rhoMax, vMax]],
  True,
   If[qL ≤ qR, qL, If[rhoL ≥ rhoCrit, qL, Max[qL, qR]]]];
```

Seeding an initial high-density block ρ=120 veh/km in the middle of an otherwise uniform road at ρ=20 veh/km produces the canonical signature of an LWR jam: a backward-travelling shock (upstream boundary of the jam) and a downstream rarefaction fan (vehicles leaving the jam into free road).

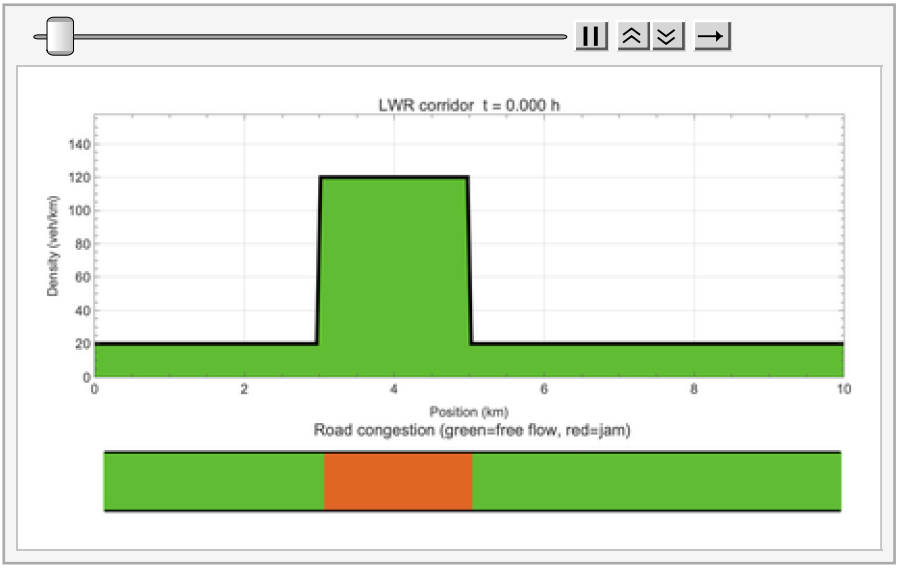

Figure 2. LWR density evolution on a 10 km corridor with an initial high–density block. The hot–coloured jam block disperses into a backward–propagating shockfront (visible travelling leftwards) and a downstream rarefaction fan; the backward propagation speed equals the slope of the shock line in the underlying space–time diagram. (The MP4 version is at Wolfram/results/anim_lwr.mp4.)

3. Adding a second equation: Payne-Whitham

A weakness of the LWR model is that velocity is always the instantaneous equilibrium value on the fundamental diagram — drivers adapt infinitely fast. Real drivers take a second or two to react, and they anticipate the state of traffic ahead. Payne (1971) and Whitham (1974) independently proposed a second equation — a momentum equation — in which velocity relaxes with time constant τ towards its equilibrium $V_e(\rho)$ and is pushed by a pressure-like term ${c_0}^2$ representing anticipation:

$$\frac{\partial v}{\partial t} + v\,\frac{\partial v}{\partial x} = \frac{V_e(\rho) - v}{\tau} - \frac{{c_0}^2}{\rho}\,\frac{\partial \rho}{\partial x}$$

coupled to the same continuity equation as LWR. The system is now second-order and supports stop-and-go waves: velocity is no longer slaved to density. In our numerical experiment we seed a congested block on an otherwise moderate-density corridor — the kind of configuration you get at an on-ramp merge — and watch the oscillatory structure develop.

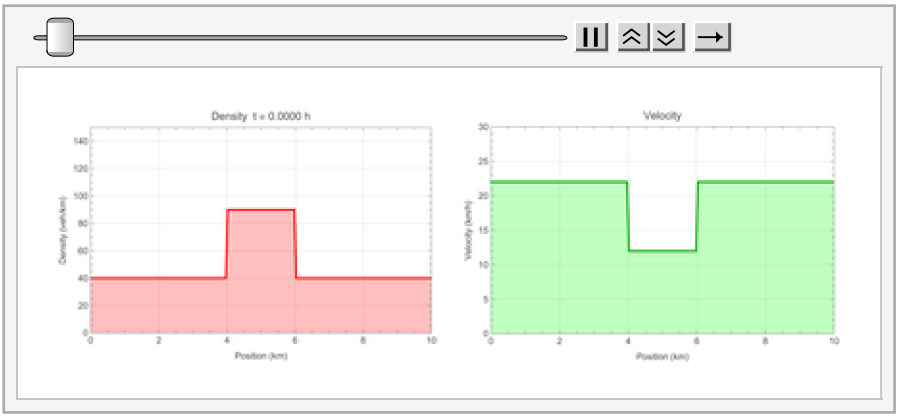

Figure 3. Animated Payne–Whitham run near a merge zone. Unlike LWR the velocity field can drop below equilibrium transiently, producing stop–and–go oscillations rather than a clean shock. (MP4: Wolfram/results/anim_payne_whitham.mp4.)

4. The microscopic view: Intelligent Driver Model

Zooming in to the level of an individual vehicle — indexed α — the Intelligent Driver Model (Treiber, Hennecke & Helbing, 2000) is a second-order ODE for the longitudinal dynamics. It is written as

$$\dot{v}_\alpha = a\left[1 - \left(\frac{v_\alpha}{v_0}\right)^{\delta} - \left(\frac{s^*(v_\alpha,\ \Delta v_\alpha)}{s_\alpha}\right)^2\right]$$

where v_α and s_α are the speed and gap to the vehicle in front, Δv_α is the approach rate, and the desired gap is

$$s^*(v,\ \Delta \mathrm{v}) = s_0 + v\,T + \frac{v\,\Delta \mathrm{v}}{2\sqrt{a\,b}}$$

The constants have direct physical meaning: v_0 is the desired free-flow speed (typically 15 m/s on an urban road), s_0 is the minimum bumper gap at standstill (2 m), T is the safe time headway to the leader (1.5 s), and a, b are the maximum acceleration and comfortable braking (1 and 1.5 m/s ^{2}). The acceleration exponent δ (typically 4) controls how sharply a driver eases off the accelerator as they approach their desired speed.

Running 50 IDM vehicles on a 1 km circular road, with a single-vehicle speed perturbation, reproduces one of the most pedagogically useful results in all of traffic science: the *phantom jam*. The perturbation is not absorbed; instead it amplifies through the platoon into a persistent backward-propagating jam that has no visible external cause. A road that was a fraction away from instability collapses into a stop-and-go wave.

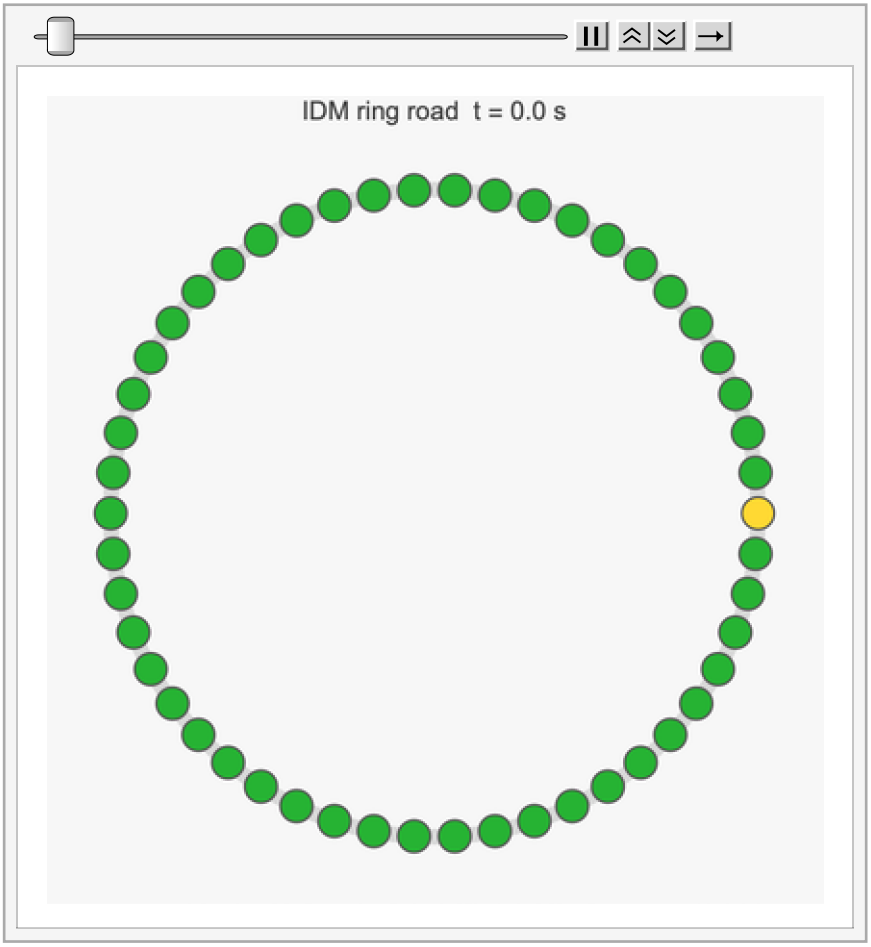

Figure 4. 50 IDM vehicles on a 1 km ring road. One vehicle is seeded at half speed at t=0; the instability propagates backward around the ring and crystallises into a persistent stop–and–go wave. Each disk is a vehicle coloured by its instantaneous speed (green = free flow, red = stopped). (MP4: Wolfram/results/anim_idm.mp4.)

5. The simplest car-following that jams: Bando OVM

Even simpler than IDM, the Optimal Velocity Model (Bando et al. 1995) relaxes speed towards a target velocity that depends only on the gap to the vehicle ahead:

$$\dot{v}_\alpha = \kappa \left[V_{\mathrm{opt}}(s_\alpha) - v_\alpha \right]$$

with the standard choice

$$V_{\mathrm{opt}}(s) = \frac{v_{\max}\left(\tanh\left(\frac{s}{s_c} - 2\right) + \tanh(2)\right)}{1 + \tanh(2)}$$

The model has a single stability parameter, the sensitivity κ. For κ above a critical value the uniform-flow solution is stable; below it, uniform flow is linearly unstable via a Hopf bifurcation and the system collapses into stop-and-go waves. This is the cleanest mathematical explanation for why phantom jams exist: they are the attractor of a dynamical bifurcation.

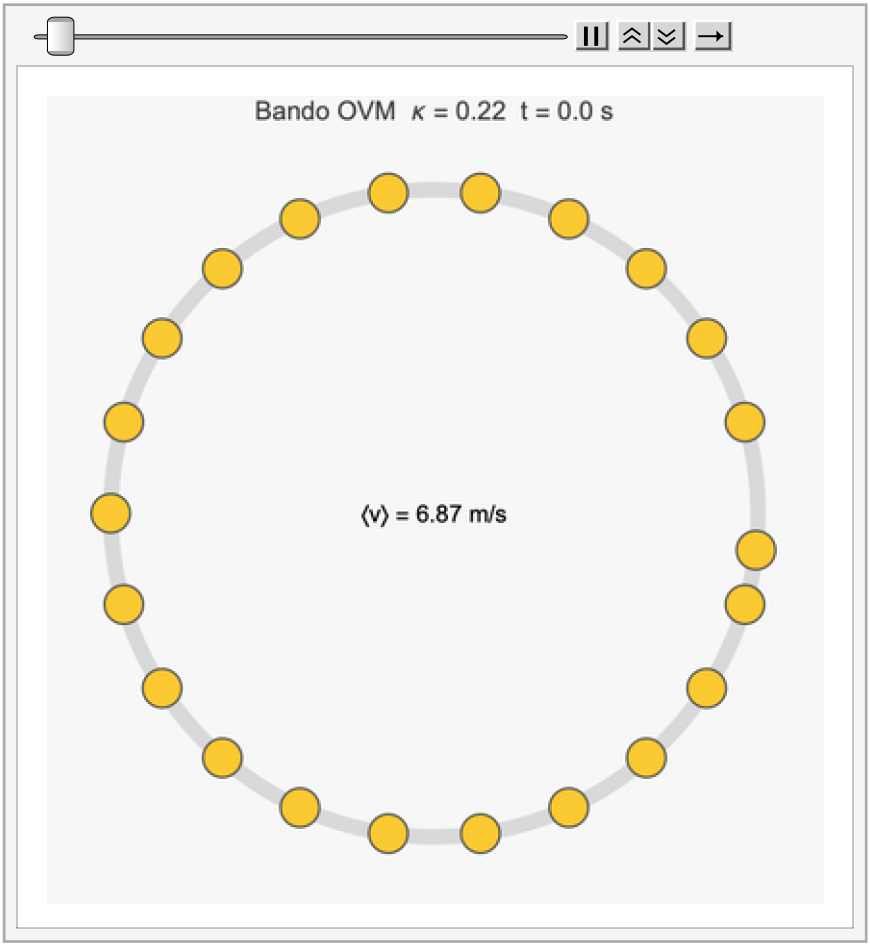

Figure 5. Animated Bando OVM on an 800 m ring with 40 vehicles, seeded by a 5 m position perturbation on one vehicle. The ring starts in near-uniform flow; the perturbation rotates around the ring, grows via the Hopf instability, and breaks the symmetry into a persistent stop-and-go wave. The mean-speed readout in the centre collapses from ~13 m/s to under 1 m/s as the instability saturates. (MP4: Wolfram/results/anim_bando.mp4.)

6. A stochastic cellular automaton: Nagel-Schreckenberg

Nagel and Schreckenberg (1992) gave the minimal CA that reproduces realistic traffic dynamics. Space is discretised into 7.5 m cells, time into 1 s steps, and velocity is an integer in $[0, v_{\max}]$. At each step every vehicle applies four rules:

$$(1)\ \text{accelerate} : v \to \min(v+1,\ v_{\max})$$

$$(2)\ \text{brake} : v \to \min(v,\ d-1)$$

$$(3)\ \text{randomise} : \text{with probability } p,\ v \to \max(v-1,\ 0)$$

$$(4)\ \text{move} : x \to x+v$$

where d is the number of empty cells to the next vehicle. The stochastic slow-down rule (3) is essential: without it the model reduces to a deterministic CA and does not jam spontaneously. With it, the space-time diagram spontaneously generates backward-propagating "wave" bands, matching the LWR and IDM results from a completely different starting point.

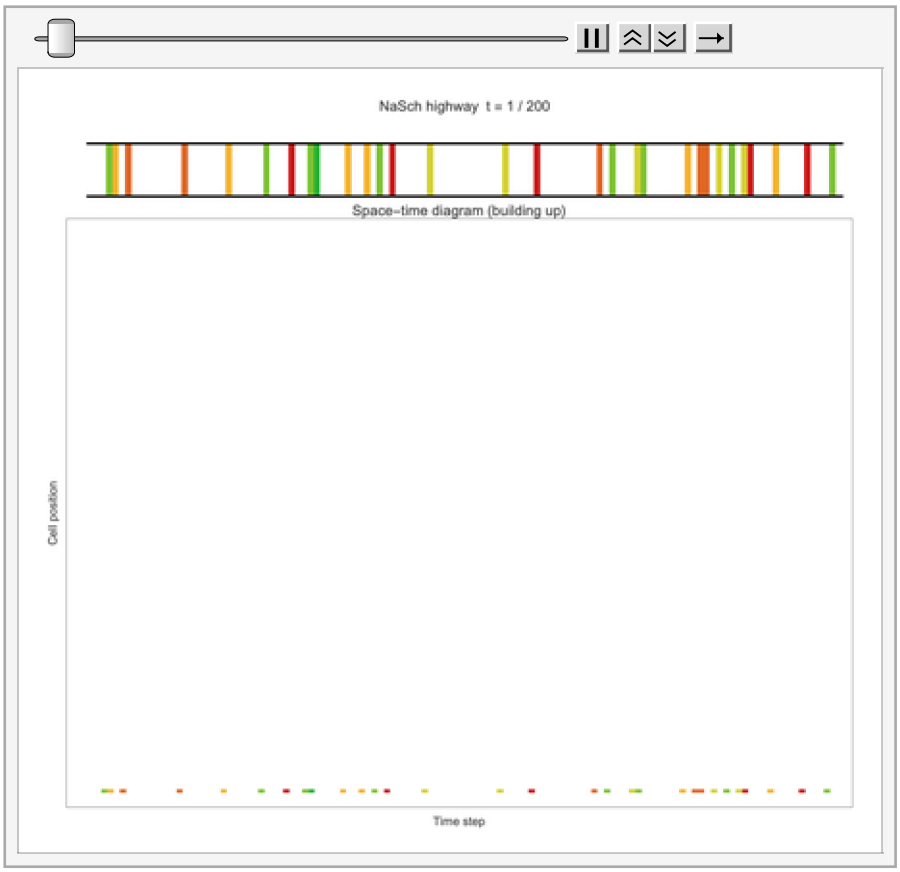

Figure 6. 100 NaSch vehicles on a 500–cell periodic lattice with randomisation probability p=0.3 and vMax=5 cells/step. Top panel: current positions, coloured by speed. Bottom panel: the space–time diagram grows line by line, and backward–propagating jam waves appear as diagonal dark bands. (*MP4: Wolfram/results/anim_nasch.mp4.*)

Extending to two lanes with a symmetric lane-change rule (incentive: would have to brake in current lane; safety: sufficient gap ahead and behind in the target lane; target cell free) recovers the classic multi-lane redistribution.

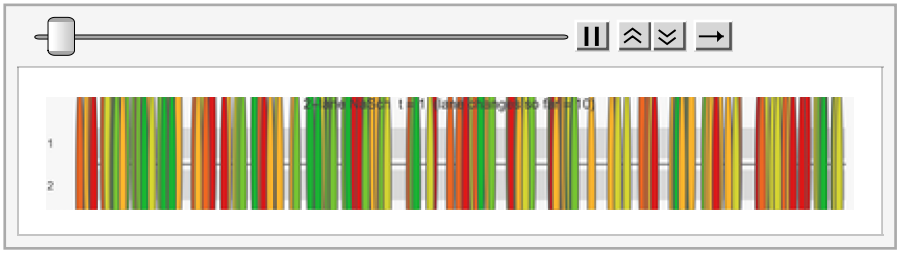

Figure 7. Two–lane NaSch with 150 vehicles (*75 per lane initially*). *The running counter in the title shows cumulative lane–change events; each occurs when a vehicle would otherwise have to brake but the other lane satisfies the gap–acceptance criteria.* (*MP4: Wolfram/results/anim_nasch_2lane.mp4.*)

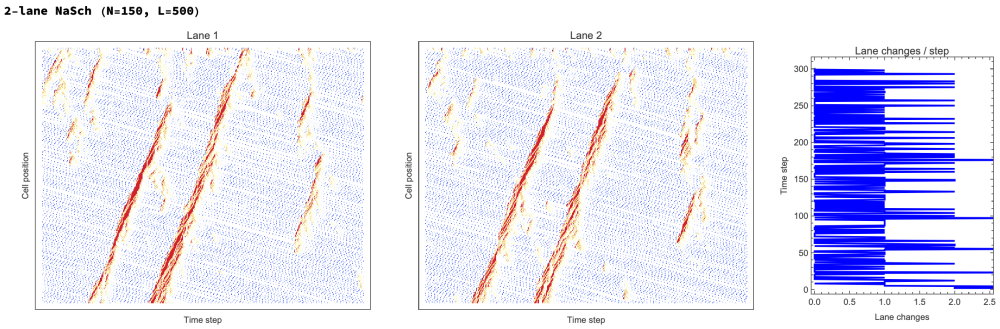

Figure 8. Static space–time view of the same 2-lane run. Left two panels: per-lane space–time diagrams. Right: number of lane changes per step. Peaks in lane-change activity coincide with density imbalances between the two lanes.

7. Intersections as queues: M/D/1

A signalised intersection looks like a server that alternates between available (green) and unavailable (red). If vehicle arrivals are Poisson with rate λ and service during the green phase is deterministic at rate μ, the steady-state queue length is given by the Pollaczek–Khinchine formula for an M/D/1 queue,

$$L_q = \frac{\rho^2}{2\,(1-\rho)}$$

with the same-shape result for the waiting time:

$$W_q = \frac{\rho}{2\,\mu\,(1-\rho)}$$

both diverging as utilisation $\rho = \frac{\lambda}{\mu}$ approaches 1. Traffic engineers read the practical threshold $\rho = 0.85$ as the onset of severe congestion. On a tandem of n signalised intersections along a corridor we approximate total delay by summing the per-intersection delays (a Jackson-network approximation that ignores platoon-preservation between signals but is accurate enough for first-pass planning).

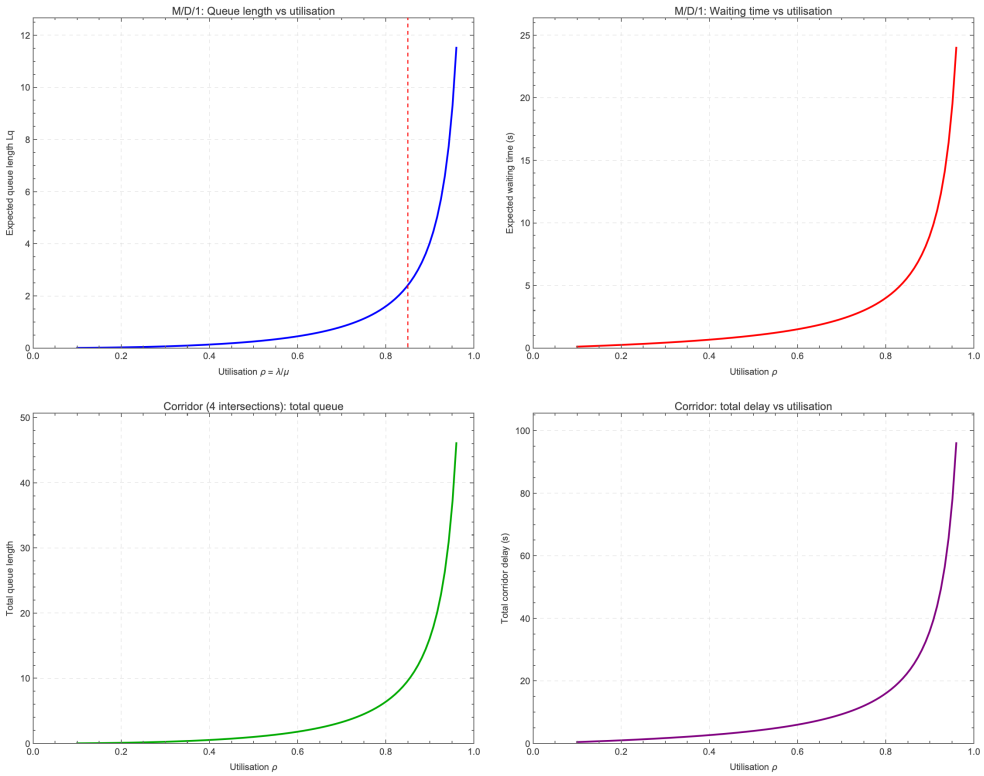

Figure 9. M/D/1 queueing on a four-intersection corridor. Top: per-intersection queue length (left) and waiting time (right) as a function of utilisation; both blow up as rho -> 1. Bottom: total queue and total delay for a 4-intersection tandem. The red dashed line marks the rho=0.85 severe-congestion onset.

8. Optimising the signal timing

Given a corridor of signals at which arrivals vary by intersection, what green times minimise total delay, subject to a shared green-time budget? Delay at each intersection combines the M/D/1 waiting term above with Webster's uniform delay for the fraction of the cycle spent red,

$$d_{\text{unif}} = \frac{(C-g)^2}{2\,C}$$

where C is cycle length and g is green time at the intersection. Minimising the sum of these over green times subject to a total budget is a smooth convex problem and `FindMinimum[..., Method -> "InteriorPoint"]` solves it in milliseconds. On a five-intersection Union Street-like corridor with arrival rates (0.08, 0.15, 0.22, 0.17, 0.06), the optimum re-allocates green from the two quiet intersections to the central busy one and cuts total delay by ~8.4 %.

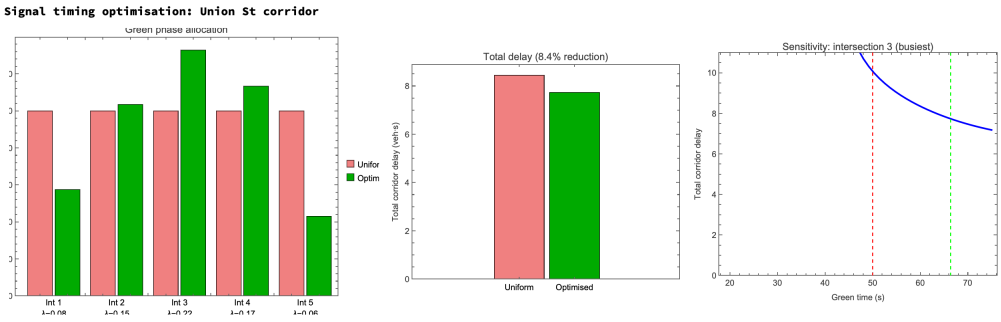

Figure 10. Webster-style green-time optimisation. Left: uniform (50 s each) versus optimised green phase allocation across the five intersections. Middle: total delay drops from 8.44 to 7.73 veh s (8.4% reduction). Right: sensitivity at the busiest intersection showing the U-shape that gives rise to the optimum.

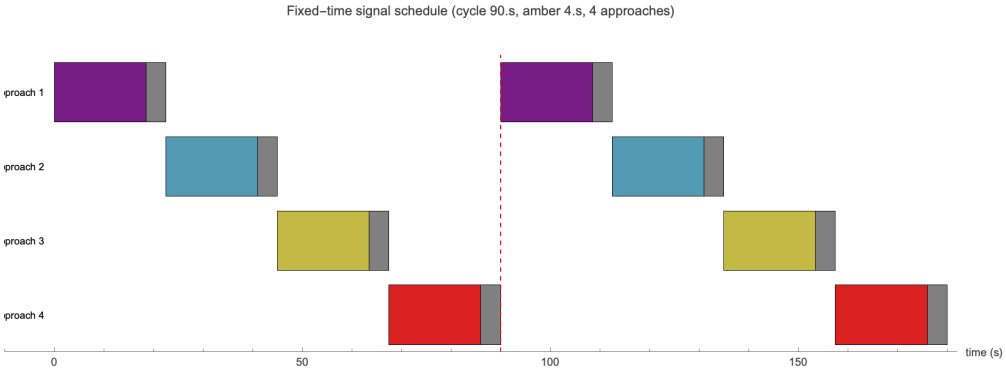

Figure 11. Gantt chart of a 90 s cycle / 4 s amber / 4-approach round-robin schedule. Each approach gets 18.5 s of green per cycle, separated by amber intergreens. The controller is the building block for signal control in the Aberdeen multi-agent simulation further down.

9. Network-level assignment: Wardrop equilibrium

Zooming back out: at the scale of a whole city, the question is no longer "what does one link look like?" but "how do drivers distribute themselves across the network?". The standard answer is Wardrop's first principle (1952): at equilibrium, all used paths between the same origin-destination pair have equal cost; no driver can unilaterally improve by switching routes. Formally, for all paths p, q used by the same OD pair,

$$c_p = c_q \quad \forall \text{ used paths } p,\ q$$

Beckmann (1956) showed that Wardrop equilibrium link flows are the unique minimiser of the convex program

$$\min_{x} \sum_{e \in E} \int_0^{x_e} t_e(\omega)\, d\omega$$

where x_e is the flow on link e and $t_e(\cdot)$ is the increasing link-cost function. The universal practical choice is the BPR formula of the US Bureau of Public Roads,

$$t_e(x_e) = t_e^0 \left[1 + \alpha \left(\frac{x_e}{C_e}\right)^{\beta}\right]$$

with t_e^0 the free-flow travel time, C_e the capacity, and tuning parameters $\alpha = 0.15$, $\beta = 4$. The objective is smooth and strictly convex, so `FindMinimum` solves it in a handful of iterations.

On a hand-curated 15-node Aberdeen network (Haudagain, Bridge of Don, Bridge of Dee, Union Street / Market Street, Anderson Drive, AWPR junction, etc.) with four origin-destination pairs

summing to 8 500 veh/h, the solver identifies Union Street and Anderson Drive (south) as the dominant bottlenecks — saturation values of 3.0× and 2.4× capacity respectively, matching the real-world experience of commuters.

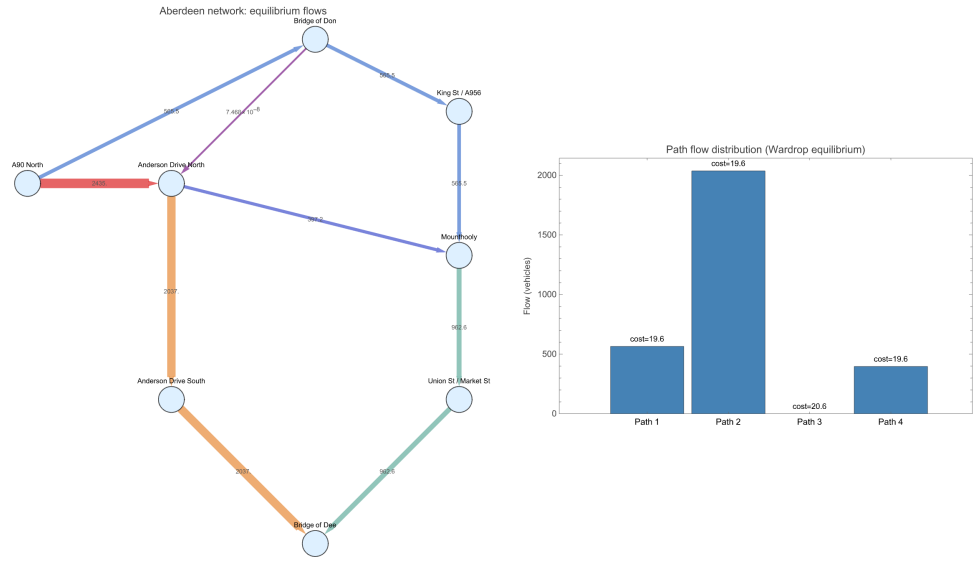

Figure 12. Wardrop–equilibrium flow on a toy 8–node Aberdeen–inspired network. Left: network flow map; edge thickness and colour encode flow. Right: path–flow distribution; all used paths (1, 2, 4) end up with approximately equal costs ≈19.6 while the unused path 3 has higher cost – exactly Wardrop's principle.

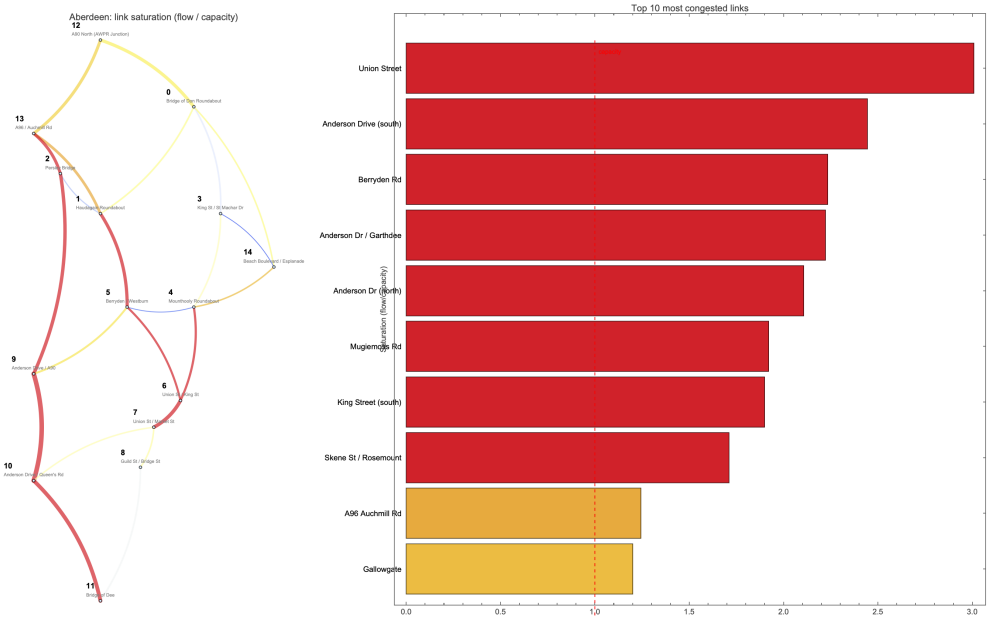

Figure 13. Equilibrium on the realistic 15–node Aberdeen network. Left: saturation map of the 23 roads; the thickest red edge is Union Street. Right: top ten most–congested links, with Union Street at 3.0x capacity followed by Anderson Drive (south) and Berryden Road.

10. Time-varying demand

Real rush-hour demand is not a single number; it is a Gaussian-ish profile that builds up, peaks, and decays. We therefore run the Wardrop solve period by period, with some fraction of each period's flow carrying over as residual congestion that reduces the effective capacity for the next period:

```
effectiveCap[e, t + 1] = Max[cap[e] - 0.3 residual[e, t], 0.5 cap[e]];
(* then solve Beckmann on (fft, effectiveCap) with that period's demand *)
```

This is a very coarse approximation to full Dynamic Traffic Assignment but captures the right qualitative behaviour: flows on the shortest route pile up first, spill over to secondary routes as the fastest route saturates, and decay asymmetrically on the way out of the peak because residual congestion persists.

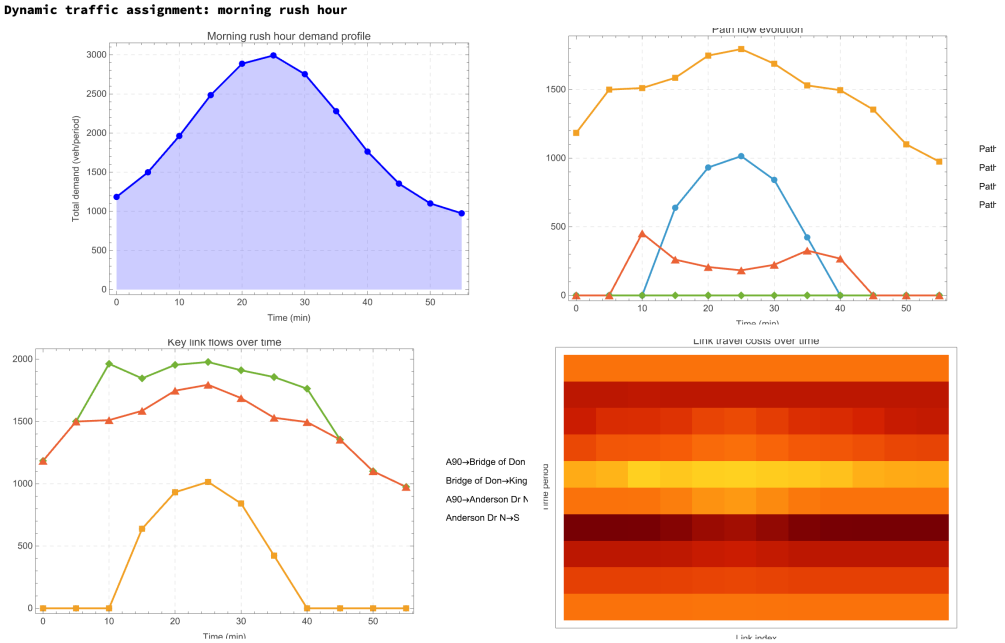

Figure 14. 12-period dynamic assignment (period length 5 min) on the toy Aberdeen network. Top-left: Gaussian-shaped demand profile peaking mid-morning. Top-right: path-flow evolution. Bottom-left: flow on four key links. Bottom-right: link-cost heatmap showing congestion spread and persistence.

11. The full case study: Aberdeen from OpenStreetMap

All the ingredients above stitch together to produce a realistic multi-agent simulation of the full Aberdeen drivable road network. The network is parsed from an Overpass API JSON extract (council-area-wide query) by filtering for drivable `highway=*` tags and splitting each way at intersection nodes, which leaves an 11 925-node, 26 453-edge directed graph after restricting to the largest strongly-connected component. 151 OSM-tagged traffic-signal junctions get a fixed-time controller with a 60 s cycle, 4 s amber, and a randomised offset. ~1000 edges tagged `junction=roundabout` enforce gap acceptance via a TRL 281-style critical headway.

Every vehicle carries a full IDM car-following state (a=2, b=3, s0=2, T=1.2, $\delta = 4$, with the velocity-difference term). Origin-destination pairs are sampled by node degree so trips cluster at major intersections. Routes are computed on a graph whose edge weights are length times a per-highway-

class factor (0.5 for motorway, 1.2 for residential, …) times a frozen per-edge random factor in [0.8, 1.5], giving genuine route diversity without rebuilding the graph on every call. Entries are staggered over the first 60 s, and at each junction there is a 10% chance of a probabilistic detour. Within 15 m of the stop line a red signal or a blocked roundabout caps the vehicle's speed-limit-for-IDM at a distance-proportional value, causing the IDM to decelerate naturally towards the stop.

```
Get["IntersectionControl.wl"];
Get["AberdeenFullCity.wl"];
net = AberdeenFullCity`LoadFullAberdeen[];
sim = AberdeenFullCity`SimulateFullAberdeen[net,
 "nVehicles" → 600, "nFrames" → 120, "T" → 360];
AberdeenFullCity`PlotFullAberdeen[sim, "aberdeen_full.png"];
AberdeenFullCity`AnimateFullAberdeen[sim, "aberdeen_full.gif"];
```

On an M-class laptop this pipeline takes about 30 s for the simulation and a minute for the GIF export. The final frame at $t = 360\ s$ shows the Aberdeen Western Peripheral Route on the outer ring, heavy flows along the A90 down the east coast and into the city, and congestion at the expected pinch points.

Figure 15. Animated full-city simulation (600 vehicles, 12 fps, 36 frames). Signal-controlled junctions impose the expected wave-structure on the arterial roads. The AWPR ring is clearly visible on the west/north perimeter and carries the lion's share of through-flow, exactly as intended when it opened in 2018. (*MP4: Wolfram/results/aberdeen_full.mp4.*)

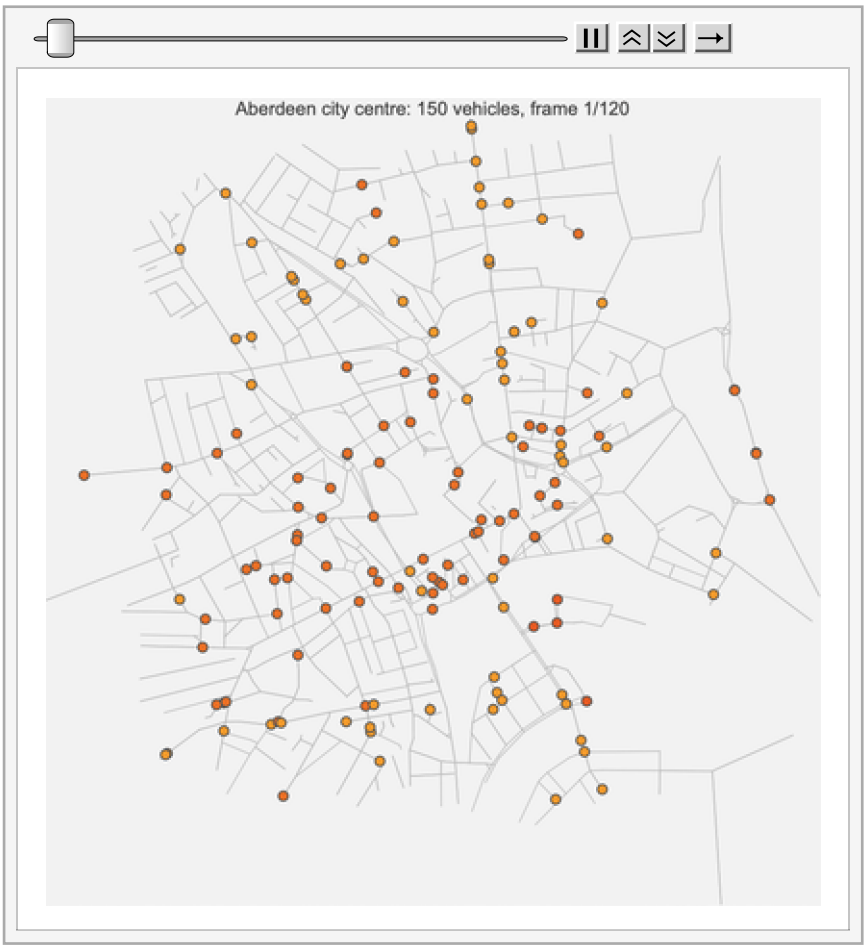

Figure 16. Zoomed-in view: 150 vehicles on the 800 m radius city-centre extract (1397 drivable nodes). Union Street cuts diagonally through the middle; Marischal College is at the centre. At this scale individual vehicle interactions and queue build-up at intersections are very visible. (MP4: Wolfram/results/aberdeen_city.mp4.)

12. Implementation notes

A few Wolfram-Language design choices that make the code short and fast:

• Each model lives in its own `.wl` package under `Wolfram/` with a `Simulate<Name>[opts]` and `Plot<Name>[result]` entry point. The top-level `RunAll.wls` runs every model in ~40 s end to end.
• IDM, Bando, and LWR use vectorised `RotateLeft` to get the leader-follower gaps in one call instead of a Python-style loop.
• The NaSch lane-changing rule uses a `Dispatch`-backed membership test for occupied target cells, which makes the inner loop practically constant-time.
• Both assignment models (`NetworkAssignment`, `DynamicAssignment`) use `FindMinimum[{obj, constraints}, vars, Method -> "InteriorPoint"]` for the Beckmann minimisation, exploiting that it is a smooth strictly convex problem.
• The full-city simulator builds a single hierarchy-weighted graph once with per-edge random multipliers in $[0.8, 1.5]$, giving frozen but distinct routes per vehicle. Shortest paths then use Wolfram's `FindShortestPath` on this one graph; the full 600-vehicle run never touches the Graph[] constructor in the inner loop.
• Signal phase logic is a tiny data-driven `Association` keyed by node: `SignalIsGreen[ctrl, edge, simTime]` is a handful of `Mod` / `MemberQ` calls, not a class hierarchy.

• Animations use `Export[path, Table[frame, ...], "DisplayDurations" -> ...]` — no external ImageMagick or ffmpeg call is needed.

---

13. References

• M. J. Lighthill & G. B. Whitham (1955), "On kinematic waves II: A theory of traffic flow on long crowded roads", *Proc. Roy. Soc. A* 229:317–345.

• M. Treiber, A. Hennecke & D. Helbing (2000), "Congested traffic states in empirical observations and microscopic simulations", *Phys. Rev. E* 62:1805.

• M. Treiber & A. Kesting (2013), *Traffic Flow Dynamics: Data, Models and Simulation*, Springer — the canonical modern textbook on the whole hierarchy.

• K. Nagel & M. Schreckenberg (1992), "A cellular automaton model for freeway traffic", *J. Phys. I France* 2:2221.

• M. Bando, K. Hasebe, A. Nakayama, A. Shibata & Y. Sugiyama (1995), "Dynamical model of traffic congestion and numerical simulation", *Phys. Rev. E* 51:1035.

• J. G. Wardrop (1952), "Some theoretical aspects of road traffic research", *Proc. ICE* 1(3):325–378.

• M. J. Beckmann, C. B. McGuire & C. B. Winsten (1956), *Studies in the Economics of Transportation*, Yale U.P.

• F. V. Webster (1958), "Traffic signal settings", HMSO Road Research Technical Paper 39.

• Source code, data, and more animations: github.com/mthiel74/TrafficJAms.